Vendredi 20/11, matin — Copier sans modifier : le spread

Kevin Bardoux

vendredi 20 novembre 2026

Table des matières

Objectifs 2
Veille — Les nouveautés de JavaScript (30 min) 2
En groupe (15 min) 2
Mise en commun et journal de veille (15 min) 3
Matin — Le spread 3
Rappel (5 min) 3
Le spread (15 min) 3
Le spread, en image : deux étiquettes, ou deux objets 4
Le spread : une copie à côté 5
Le spread, l'exemple minimal 5
Le spread — attention et vérification 5
Pas à pas ensemble — Ajouter sans modifier (25 min) 6
Variante à faire seuls 8
Enregistrer et envoyer (2 min) 8
Exercices — Le spread 8
Palier 1 — Ajouter sans modifier (15 min) 8
Palier 2 — Changer la ville, mettre à jour un musée (35 min) . 9
Palier 3 — Retirer sans modifier (25 min) 10
Enregistrer et envoyer (2 min) 11
À retenir 11
Si le temps le permet 11
Débrief 11
Sources 11

Note

Compétences visées : C4 · apprendre en continu (veille) · communiquer en anglais **Livrable** : une entrée de plus dans votre journal de veille ; le pas à pas `pas-a-pas-spread` envoyé sur GitHub, et `node --test k4-ajouter.test.js` au vert.

! Le minimum du jour

- Une entrée de plus dans votre journal de veille.
- Le pas à pas « Ajouter sans modifier », fait avec le groupe.
- Le palier 1 : `node --test k4-ajouter.test.js` au vert.

Les paliers 2 et 3, puis la rubrique « Si le temps le permet », sont la suite naturelle, pour ceux qui avancent.

Objectifs

- Savoir qui fait évoluer JavaScript, et où lire ses nouveautés.
- Copier un tableau ou un objet avec `...`, en changeant ou en ajoutant un élément, sans modifier l'original.
- Distinguer une copie d'une deuxième étiquette sur le même objet.

Comment se passe la matinée. La veille, de 9 h 05 à 9 h 35. Puis une seule notion : le *spread*. Il sert toute l'année : en semaine 12, React mettra à jour ses données exactement de cette façon.

Veille — Les nouveautés de JavaScript (30 min)

En groupe (15 min)

1. **Groupe 1** : le site du TC39 : qui fait évoluer JavaScript ? Comment ?
2. **Groupe 2** : la liste des propositions terminées.
3. Groupe 2 : trouvez celle d'où vient `toSorted`, et son année.
4. Dans chaque groupe : un lecteur, un secrétaire, un porte-parole.

Sources en anglais : le site du TC39, le comité qui fait évoluer la norme ECMAScript, le nom officiel de JavaScript (TC39, s. d.-b), et

sa liste des propositions terminées, avec leur année de publication prévue (TC39, s. d.-a). Le groupe 2 cherche `toSorted`, vu hier : il vient d'une proposition au nom anglais (indice : *copy*). Puis il en choisit une autre, de la même liste, et l'explique avec ses mots.

Les rôles : le lecteur lit la page à voix haute ; le secrétaire note les réponses ; le porte-parole présente. Celui qui préfère ne pas parler devant le groupe prend le rôle de secrétaire.

Mise en commun et journal de veille (15 min)

1. Chaque porte-parole présente ce que son groupe a trouvé (3 min).
2. Seul : ajoutez l'entrée du jour à votre journal de veille.

Réussi si : votre journal de veille a une entrée datée du 20/11, avec la source (le lien), ce que vous avez appris en deux phrases, et le nom d'une proposition terminée.

Matin — Le spread

Rappel (5 min)

1. Que fait `sort` au tableau qu'il trie ?
2. Que renvoie `[3, 8].reduce((s, n) => s + n, 0)` ?
3. Combien de tests au plus pour un exercice ?

Trois questions courtes, sans regarder vos notes : la veille, cette semaine, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

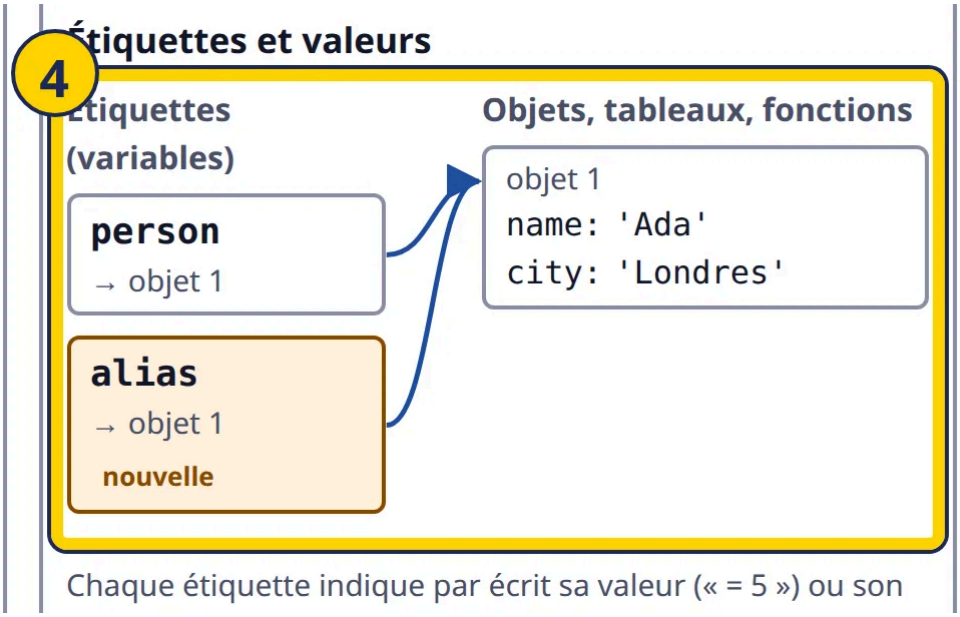
Le spread (15 min)

- **Pourquoi** : changer une donnée sans abîmer l'original.
- **Spread** : `...` recopie les éléments dans un nouveau tableau ou objet.
- **Immutabilité** : on ne modifie pas, on crée une copie modifiée.

Pourquoi. Depuis le début de la semaine, aucune fonction ne modifie le tableau qu'elle reçoit : `map`, `filter`, `toSorted` renvoient un nouveau tableau. Mais comment **ajouter** un élément, ou **changer** une propriété d'un objet, sans modifier l'original ? Avec `push` ou une affectation, on modifie l'original.

Définition. La syntaxe de décomposition, ou *spread*, `...`, développe les éléments d'un tableau ou les propriétés d'un objet dans un **nouveau** tableau ou un **nouvel** objet (MDN Web Docs, s. d.) : `[...list, 'c']` crée un nouveau tableau ; `{ ...museum, city: 'METZ' }`, un nouvel objet. Ne jamais modifier l'original, c'est l'immutabilité.

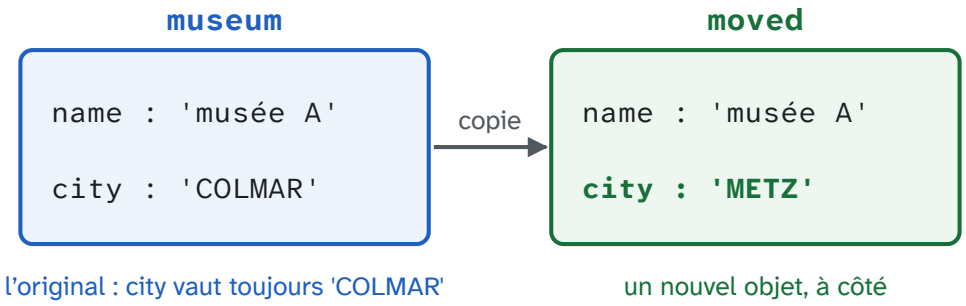
Le spread, en image : deux étiquettes, ou deux objets



Ce que montre l'image : après `const alias = person;`, les **deux** étiquettes, `person` et `alias`, désignent le **même** objet. Changer `alias.city`, c'est changer l'objet de `person`. Avec `{ ...person, city: 'Rome' }`, un deuxième objet est créé : `person` ne bouge plus. Rejouez l'exemple dans le visualiseur pas à pas, avec **Ligne suivante**.

Le spread : une copie à côté

```
const moved = { ...museum, city: 'METZ' };
```



...museum recopie les propriétés ; city: 'METZ' remplace la valeur dans la copie seulement. Deux objets, deux étiquettes.

Figure 1. – Le spread crée un nouvel objet à côté de l'original, qui ne change pas.

Le spread, l'exemple minimal

```
const names = ['musée A', 'musée B'];
const withNew = [...names, 'musée C'];
// names ne change pas ; withNew a trois éléments

const museum = { name: 'musée A', city: 'COLMAR' };
const moved = { ...museum, city: 'METZ' };

const visits = [51775, 3165];
const sorted = [...visits].sort((a, b) => a - b);
// la deuxième façon de trier sans modifier
```

Le spread — attention et vérification

> ⚠ Attention
>
> `const copie = original` ne copie rien : c'est une deuxième étiquette sur le **même** tableau ou le même objet. Pour copier : `[...original]` ou `{ ...original }`.

- **Vérifiez** : `const b = [...a]; b.push(1);` : `a` a-t-il changé ?
- Et avec `const b = a; b.push(1);` ?

Pas à pas ensemble — Ajouter sans modifier (25 min)

- Le formateur code, vous tapez en même temps, une étape à la fois.
- Code et commentaires : `//` au-dessus de chaque ligne.
- Dossier : `pas-a-pas-spread`, fichier `copies.js`.
- Perdu ? Dossier `rattrapage/spread/`, une étape à la fois.

Palier 1 : guidé. Ouvrez `pas-a-pas-spread/copies.js`. Dans le terminal : `cd pas-a-pas-spread`. L'étape 4 montre une deuxième façon de faire ce que fait `toSorted` (hier) : trier une **copie**.

Avant de commencer. Lancez `node copies.js`, depuis `pas-a-pas-spread`. **Vous devez voir** : les données de départ, sans erreur. Chaque étape ajoute ses lignes à la fin du fichier, et sa sortie à la fin de la console.

Étape 1 — Copier un tableau. À la fin du fichier, copiez `names` avec `...`, puis ajoutez un nom à la copie seulement.

Avant d'exécuter : `push` sur `copy` : `names` va-t-il changer ?

```
// Spread : ... recopie les éléments dans un NOUVEAU tableau
const copy = [...names];
// push ne touche que la copie
copy.push('musée Arthur Rimbaud');
// Affiche l'original, puis la copie
console.log(names, copy);
```

Enregistrez, puis lancez `node copies.js` dans le terminal, depuis `pas-a-pas-spread`.

Vous devez voir, à la fin de la sortie :

```
[ 'musée Unterlinden', "écomusée d'Alsace" ] [ 'musée Unterlinden', "écomusée d'Alsace", 'musée Arthur Rimbaud' ]
```

Perdu ? Copiez `rattrapage/spread/etape-1/copies.js` à la place de votre `copies.js`, puis repartez avec le groupe.

Étape 2 — Ajouter à la fin. Créez un nouveau tableau : les éléments de `names`, puis un nom de plus.

Avant d'exécuter : `names` aura-t-il trois éléments ?

```
// Spread : un nouveau tableau, avec un nom en plus à la fin
const withNew = [...names, "musée de l'Ardenne"];
// L'original n'a pas changé
console.log(names, withNew);
```

Enregistrez, puis lancez `node copies.js` dans le terminal, depuis `pas-a-pas-spread`.

Vous devez voir, à la fin de la sortie :

```
[ 'musée Unterlinden', "écomusée d'Alsace" ] [ 'musée Unterlinden', "écomusée d'Alsace", "musée de l'Ardenne" ]
```

Perdu ? Copiez `rattrapage/spread/etape-2/copies.js` à la place de votre `copies.js`, puis repartez avec le groupe.

Étape 3 — Copier un objet, une propriété changée. Créez `moved` : une copie de `museum`, avec une autre ville.

Avant d'exécuter : quelle ville `museum` va-t-il garder ?

```
// Un objet : un musée et sa ville
const museum = { name: 'musée Unterlinden', city: 'COLMAR' };
// Spread : un NOUVEL objet, avec une autre ville
const moved = { ...museum, city: 'METZ' };
// L'original garde sa ville
console.log(museum, moved);
```

Enregistrez, puis lancez `node copies.js` dans le terminal, depuis `pas-a-pas-spread`.

Vous devez voir, à la fin de la sortie :

```
{ name: 'musée Unterlinden', city: 'COLMAR' } { name: 'musée Unterlinden', city: 'METZ' }
```

Perdu ? Copiez `rattrapage/spread/etape-3/copies.js` à la place de votre `copies.js`, puis repartez avec le groupe.

Étape 4 — Trier une copie : la deuxième façon. Copiez `visits` avec `...`, puis triez la copie avec `sort`.

Avant d'exécuter : `visits` va-t-il être modifié ?

```
// Visites de trois musées
const visits = [51775, 3165, 29212];
// Spread puis sort : on trie la copie, pas l'original
const sorted = [...visits].sort((a, b) => a - b);
// Deuxième façon de trier sans modifier le tableau reçu
console.log(visits, sorted);
```

Enregistrez, puis lancez `node copies.js` dans le terminal, depuis `pas-a-pas-spread`.

Vous devez voir, à la fin de la sortie :

```
[ 51775, 3165, 29212 ] [ 3165, 29212, 51775 ]
```

Perdu ? Copiez `rattrapage/spread/etape-4/copies.js` à la place de votre `copies.js`, puis repartez avec le groupe.

Variante à faire seuls
Pour ceux qui ont validé l'étape avant les autres, dans le même fichier. Créez `updated` : une copie de `museum` avec `total: 165484` en plus. Affichez `museum` et `updated`. **Réussi si** : la console affiche `museum` sans `total`, `updated` avec.

Enregistrer et envoyer (2 min)
Depuis `~/dwwm/dwwm-tp/semaine-07` : `git status`, `git add pas-a-pas-spread`, `git commit -m "Ajoute le pas à pas spread"`, puis `git push`. **Réussi si** : `git status` affiche `rien à valider, la copie de travail est propre`.

Exercices — Le spread

Palier 1 — Ajouter sans modifier (15 min)

1. `k4-ajouter.js` : un nouveau tableau, avec un élément à la fin.

2. **Réussi si** : `node --test k4-ajouter.test.js` au vert.

Palier 1 : guidé. Durée : 15 min.

1. Ouvrez `k4-ajouter.js`. `withItem(list, item)` renvoie un **nouveau** tableau : les éléments de `list`, puis `item`.
2. Écrivez `return [...list, item];`.
3. Lancez `node --test k4-ajouter.test.js`. Essayez aussi avec `list.push(item)` : le test « la liste reçue n'est pas modifiée » échoue. Remettez le *spread*.
4. Un commit.

Réussi si : `node --test k4-ajouter.test.js` affiche `ℹ fail 0`.

> ℹ Aides
>
> - **Le cours** : Vendredi matin, « Le spread » ; le pas à pas, étape 2.
> - **L'IA** : CS50.ai (palier IA 1) : demandez la différence entre `push` et `[...list, item]`, pas le code.
> - **MDN** : Syntaxe de décomposition.

Palier 2 — Changer la ville, mettre à jour un musée (35 min)

1. `k6-changer-ville.js` : un nouvel objet, avec une autre ville.
2. `k8-mettre-a-jour.js` : `map`, et un *spread* pour le musée concerné.
3. **Réussi si** : leurs tests au vert.

Palier 2 : recommandé, semi-guidé. Durée : 35 min.

1. **`k6-changer-ville`** : `withCity(person, city)` renvoie un nouvel objet identique à `person`, avec `city`. Toutes les autres propriétés sont gardées. `node --test k6-changer-ville.test.js`.
2. **`k8-mettre-a-jour`** : `renameMuseum(museums, id, newName)` renvoie un nouveau tableau où seul le musée d'identifiant `id` change de nom.
   - `map` parcourt les musées.

- Pour le musée concerné, renvoyez `{ ...museum, name: newName }` ; pour les autres, le musée tel quel.
- `node --test k8-mettre-a-jour.test.js`.

Réussi si : `node --test k6-changer-ville.test.js` et `node --test k8-mettre-a-jour.test.js` affichent `ℹ fail 0`.

> ℹ Aides
>
> - **Le cours** : Vendredi matin, « Le spread » ; Lundi après-midi, « Callback et map ».
> - **L'IA** : CS50.ai (palier IA 1) : demandez comment `map` peut renvoyer un élément changé et les autres inchangés, pas le code.
> - **MDN** : Syntaxe de décomposition, section sur les littéraux objets.

Palier 3 — Retirer sans modifier (25 min)

- `k5-retirer.js` : un nouveau tableau, sans l'élément à une position.
- **Réussi si** : `node --test k5-retirer.defi.test.js` au vert.

Palier 3 : autonome. Durée : 25 min.

`withoutIndex(list, index)` renvoie un nouveau tableau sans l'élément à la position `index`, sans modifier `list`. Il existe plusieurs façons de faire : avec le *spread* et une méthode qui prend une partie d'un tableau, ou avec une méthode de la semaine qui garde certains éléments. Cherchez dans MDN, puis choisissez.

Réussi si : `node --test k5-retirer.defi.test.js` affiche `ℹ fail 0`.

> ℹ Autonome
>
> Vous allez bloquer : c'est voulu, c'est le métier. Méthode : 1. lisez le message d'erreur en entier ; 2. cherchez-le dans la fiche « Lire un message d'erreur » ; 3. lisez la documentation ; 4. après 15 minutes bloqué sur le même point, demandez.

Aides

- **Le cours** : Vendredi matin, « Le spread » ; Mardi matin, « filter et find ».
- **L'IA** : CS50.ai (palier IA 1) : demandez quelles méthodes de tableau ne modifient pas l'original, pas le code.
- **MDN** : Array, liste des méthodes.

Enregistrer et envoyer (2 min)

Même rituel : `git status`, `git add`, `git commit -m "…"`, `git push`, depuis `semaine-07`. C'est la fin de la semaine : le `git push` est obligatoire.

À retenir

À retenir

- `[...list, x]` : un nouveau tableau ; `{ ...obj, clé: v }` : un nouvel objet.
- `const b = a` ne copie rien : deux étiquettes, un seul objet.
- **Trier une copie** : `toSorted(…)`, ou `[...list].sort(…)`.

Si le temps le permet

- Les paliers 2 et 3 non terminés.
- Le parcours d'autonomie.

Débrief

- Pourquoi `push` ne convient pas pour « ajouter sans modifier » ?
- Quelle différence entre `const b = a` et `const b = [...a]` ?

Sources

MDN Web Docs. s. d. « Syntaxe de décomposition - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/Spread_syntax.

TC39. s. d.-a. « Finished Proposals (tc39/proposals) ». Consulté le 26 septembre 2026. https://github.com/tc39/proposals/blob/main/finished-proposals.md.

TC39. s. d.-b. « TC39 - Specifying JavaScript ». Consulté le 26 septembre 2026. https://tc39.es/.